

Expressões, Atribuição e Controle

Aula 07 · Paradigmas de Linguagens de Programação

Engenharia de Software

Referência: Sebesta, Conceitos de Linguagens de Programação, 11ª ed., Caps. 7 e 8 (p. 299-361)

Slide 1

Slide de abertura da Aula 07, de Paradigmas de Linguagens de Programação (Engenharia de Software). A aula usa como referência os capítulos 7 e 8 do livro de Sebesta. Na aula passada o tema foram os tipos de variáveis; hoje o foco é como os valores são combinados (expressões), guardados (atribuição) e em que ordem as sentenças executam (controle). A aula é exploratória: a turma prevê a saída de cada código antes de executá-lo no OneCompiler.

Objetivo e ligação com o mundo real

Objetivo

O objetivo desta aula é entender **como as linguagens avaliam expressões e controlam o fluxo de execução**, para que o seu código faça exatamente o que parece fazer, **sem surpresas** de precedência, conversão ou desvio.

Para o engenheiro de software, isso significa:

- ler código alheio sem ser enganado pela indentação ou pela precedência
- evitar conversões implícitas perigosas (== do JS e do PHP)
- escolher a estrutura de controle mais legível para cada caso

Quando o fluxo sai do controle

2014 · "goto fail" (Apple)

Uma linha `goto fail`; duplicada pulava a verificação de assinatura: iOS e macOS aceitavam certificados falsos.

2008 · Tocadores Zune (Microsoft)

No último dia de um ano bissexto, um laço `while` nunca terminava, e todos os aparelhos travaram.

PHP · comparação com ==

`"0e4..." == "0e8..."` é verdadeiro; logins que comparavam hashes com == aceitavam senhas erradas.

Sebesta, 11ª ed. · Caps. 7 e 8 · p. 299-361

Slide 2

O objetivo é entender como as linguagens avaliam expressões e controlam o fluxo, para que o código faça exatamente o que parece fazer. Três casos reais mostram o custo de errar nisso: o bug "goto fail" da Apple (2014), que aceitava certificados falsos; o travamento dos tocadores Zune no último dia de um ano bissexto (2008); e logins em PHP que aceitavam senhas erradas por causa do ==. Os três são erros de estrutura de controle ou de avaliação de expressões, não de lógica de negócio.

Roteiro da aula (80 min)



Tipo de aula: exploratória. A turma **prevê a saída antes de executar**.

Exemplos: pasta `exemplos/`, na ordem dos slides. Todos rodam no navegador em onecompiler.com.

Planejamento do Encontro 2 - Sehesta Caps. 7 e 8

Slide 3

Roteiro de 80 minutos: quiz de abertura (0-10 min), expressões (10-30), conversões e curto-circuito (30-45), atribuição (45-55), seleção, iteração e goto (55-70) e exercício de encontrar o bug (70-80). A dinâmica é sempre a mesma: mostrar o código, a turma prevê a saída, executar e então explicar a regra da linguagem com o livro.

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

As respostas aparecem ao longo da aula · Sebesta Cap. 7

Slide 4

Quiz de abertura com seis trechos em Python, Java, JavaScript e C. Respostas: 1) -4 e 512; 2) 3345; 3) 3 e 52; 4) 0; 5) padrão e True; 6) true, false, true. Cada resposta é explicada ao longo da aula. Normalmente ninguém acerta todas, e é essa a motivação da aula.

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

a + b * c com a=3, b=4, c=5

da esquerda para a direita

$$(3 + 4) * 5 = 35$$

da direita para a esquerda

$$3 + (4 * 5) = 23$$

com precedência (* antes de +)

$$3 + 4 * 5 = 23$$

APL: sempre da direita para a esquerda (p. 304)

$$3 \times 4 + 5 = 27$$

Sebesta, 11ª ed. · §7.2 e §7.2.1 · p. 300-305

Slide 5

As seis questões de projeto das expressões aritméticas: precedência, associatividade, ordem de avaliação dos operandos, efeitos colaterais, sobrecarga de operadores e mistura de tipos. Com $a + b * c$ ($a=3, b=4, c=5$), o resultado depende da regra: 35 da esquerda para a direita, 23 com a precedência usual. O APL avalia sempre da direita para a esquerda, o que mostra que a precedência é convenção de projeto. Lisp dispensa a precedência usando notação prefixa.

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
<code>-2 ** 2</code>	-4	erro de sintaxe	(não há <code>**</code>)
<code>2 ** 3 ** 2</code>	512	512	—
<code>1 + 2 + "3"</code>	<code>TypeError</code>	"33"	"33"
<code>-7 / 2</code> (inteiros)	-4 (com <code>//</code>)	-3.5	-3
<code>-7 % 2</code>	1	-1	-1

Associatividade (p. 303)

Esquerda: `*` / `%` + `-`

Direita: `**` (Python, JS, Ruby, Fortran),
`++`, `--`, menos unário

Visual Basic: `^` à esquerda!

Excel

`=-2^2` dá **4**: lá o menos unário vence o `^`.

Na dúvida, use **parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Sebesta, 11ª ed. · §7.2.1.1-7.2.1.3 · p. 301-305 · exemplos/01_precedencia

Slide 6

Precedência e associatividade variam entre linguagens: `-2 ** 2` dá -4 em Python e erro de sintaxe em JavaScript; a divisão e o resto de inteiros negativos diferem entre Python, JavaScript e Java. A exponenciação associa à direita, e no Excel o menos unário vence o `^` (`=-2^2` dá 4). Na dúvida, use parênteses: eles deixam a intenção clara para o compilador e para o próximo leitor.

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Sebesta, 11ª ed. · §7.2.2 · p. 307-309 · exemplos/02_efeitos_colaterais

Slide 7

Exemplo do livro: $a = a + \text{fun1}()$, em que `fun1` altera `a` e devolve 3. O resultado é 8 ou 20 conforme a ordem de avaliação. Java garante a ordem da esquerda para a direita (8); em C a ordem não é definida pelo padrão e o gcc deu 20; em Go o resultado também foi 20. Sem efeitos colaterais, a ordem não importa (transparência referencial) e as expressões podem ser trocadas sem alterar o programa.

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1);           // 98
System.out.println("" + 'a' + 1);     // a1
x.add(y).multiply(tres); // BigDecimal: sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):          # v1 + v2
        return Vetor(self.x + outro.x,
                       self.y + outro.y)

    def __mul__(self, k):              # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)         # Vetor(4, 6)
print(v * 3)                          # Vetor(12, 18)
```

Sebesta, 11ª ed. · §7.3 · p. 309-310 e §9.11 · p. 404 · exemplos/03_sobrecarga

Slide 8

Sobrecarga de operadores: o mesmo símbolo com vários significados, como o + de Java, que soma números e concatena textos. O livro alerta para o & de C++, que é E bit a bit e também "endereço de". Java não permite sobrecarga definida pelo usuário (por isso BigDecimal usa add e multiply). Em Python, métodos especiais como __add__ e __mul__ permitem somar vetores com +. A discussão é se isso melhora a legibilidade ou esconde comportamento.

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;      // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com `a` digitado por engano no lugar de `c`, o Java converte e compila.

Sebesta, 11ª ed. · §7.4 · p. 310-313 · exemplos/04_conversoes

Slide 9

Conversões de tipos: o alargamento (int para float) costuma ser seguro, mas pode perder precisão (123456789 vira 123456792). O estreitamento perde informação, a coerção é implícita e o cast é explícito. Go, Rust, ML e F# exigem conversão explícita; C, Java e Python convertem sozinhos. O livro destaca que a coerção esconde erros, como um nome de variável digitado errado que ainda assim compila.

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + +"a" + "a" // "baNaN"

0 == ""          // true
"" == "0"        // false
0 == "0"         // true  (não é transitivo!)
0 === "0"        // false (=== não converte)
```

PHP - "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01"); // true

md5("240610708") // "0e462097431906...."
md5("QNKCDZO")  // "0e830400451993...."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com `===` e `!==`, que **não** fazem coerção.

Sebesta, 11ª ed. · §7.4.1 Coerção em expressões · p. 311-312 · exemplos/04_conversoes

Slide 10

JavaScript e PHP fazem muita coerção implícita: "5" - 2 dá 3 e "5" + 2 dá "52"; o == de JavaScript não é transitivo. No PHP, "0e123" == "0e456" é verdadeiro porque os dois textos parecem números em notação científica, e isso fez logins aceitarem hashes MD5 errados. Regra prática: compare sempre com === e !==, que não fazem coerção.

Expressões relacionais e booleanas

C · booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c);    // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c));    // 1

if (-1) printf("verdadeiro\n");    // != 0
```

Python · comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c)    # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0  0.0  ""  []  {}  None
# contam como VERDADEIRO:
# "0"  [0]  " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: if (n) com n inteiro nem compila

Sebesta, 11ª ed. · §7.5 · p. 313-315 · exemplos/05_booleanas

Slide 11

Expressões relacionais e booleanas. Em C o booleano é inteiro, e $3 > 2 > 1$ vale 0, porque $(3 > 2)$ vale 1 e $1 > 1$ é falso. Em Python a mesma expressão é encadeada e vale True. C, Python e JavaScript aceitam qualquer valor numa condição (0 e vazio contam como falso), enquanto Java, C#, Go e Rust exigem um booleano no if.

Avaliação em curto-circuito

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;           // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0 // false
s != null & s.length() > 0 // NullPointerException
```

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade // undefined, sem erro

const qtd = 0;
qtd || 10           // 10 (o 0 se perdeu!)
qtd ?? 10           // 0 (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em `(a > b) || (b++ / 3)`, o `b++` só acontece quando `a <= b`. Em Python, `0 or "padrão"` devolve "padrão": `and` e `or` devolvem um dos **operandos**.

Sebesta, 11ª ed. · §7.6 · p. 315-317 · exemplos/05_booleanas

Slide 12

Curto-circuito: o resultado é determinado sem avaliar todos os operandos. Isso protege acessos, como na busca em lista que evita ler uma posição fora do vetor, e evita erros com null em Java (&& faz curto-circuito, & não). Em JavaScript, `?.` e `??` são curtos-circuitos para null; `??` troca só null e undefined, ao contrário de `||`, que também descarta o 0. Cuidado com efeitos colaterais, que podem não executar.

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... } // compila! atribui e testa y
```

Python · atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: if (x = y) compila
Java, C#	sim, mas o if exige booleano
Python	não, exceto o operador morsa :=
Go	não: nem x++ é expressão
Rust	vale (): if x = 5 não compila

Linguagens funcionais (p. 321)

val (ML) e let (F#, Rust) dão **nome** a um valor, que não muda.

Sebesta, 11ª ed. · §7.7 · p. 317-321 · exemplos/06_atribuicao

Slide 13

Sentenças de atribuição. Em C, JavaScript e outras, a atribuição é uma expressão e devolve o valor, o que permite `sum = count = 0` mas também o erro clássico `if (x = y)`, que compila. Python não trata atribuição como expressão (exceto o operador morsa `:=`), Go e Rust também não. Python permite atribuição múltipla (`a, b = b, a`), e nas linguagens funcionais `let` e `val` apenas dão nome a um valor imutável.

Atribuição de modo misto

Java · só alargamento (p. 322)

```
double d = 3;           // ok: alargamento
// int i = 3.7;         // erro: estreitamento
int i = (int) 3.7;       // 3 (cast explícito)
long l = i;              // ok

i += 3.7;                // COMPILA: cast escondido
System.out.println(i);   // 6
```

C · coerção em qualquer direção

```
int i = 3.7;             // 3, em silêncio
char c = 300;            // 44 (só um aviso)
unsigned int u = -1;     // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100;` com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para `double`.

Sebesta, 11ª ed. · §7.8 · p. 321-322 · exemplos/06_atribuicao

Slide 14

Atribuição de modo misto. Java só permite a coerção de alargamento (`int i = 3.7` não compila; `+= 3.7` compila por causa de um cast escondido). C converte em qualquer direção, em silêncio (`int i = 3.7` vira 3, `char c = 300` vira 44). Armadilha clássica: `acertos / total * 100` com inteiros dá 0.0, porque a divisão inteira acontece antes do alargamento para `double`.

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;  
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...  
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)  
    x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (for, switch, do, foreach)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Sebesta, 11ª ed. · §8.1 · p. 328-330

Slide 15

Estruturas de controle: sequência, seleção e iteração bastam para qualquer algoritmo (Böhm e Jacopini, 1966), e o goto não é necessário. As demais estruturas (for, switch, do, foreach) existem pela facilidade de escrita. O limite é a legibilidade: quanto maior a linguagem, menos do código alheio cada pessoa reconhece.

Seleção de dois caminhos: o "else pendente"

C · a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1;          // de qual if é este else?
printf("%d\n", result);  // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };          // Rust: if é uma expressão (como em F#, p. 334)
y = x if x > 0 else 2 * x                      # Python: expressão condicional (p. 306)
```

Sebesta, 11ª ed. · §8.2.1 · p. 330-334 · exemplos/07_selecao

Slide 16

O "else pendente": em C, Java e C#, o else pertence ao if mais próximo que ainda não tem else, e a endentação é ignorada pelo compilador, então o código se comporta diferente do que sugere o recuo (result vale 1). Chaves obrigatórias (Perl, Go, Rust) eliminam o problema. Em Rust e F# o if é uma expressão que devolve valor, e Python tem a expressão condicional `x if c else y`.

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {
  case 1:
  case 3: odd += 1;
  case 2:
  case 4: even += 1;
  default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {
  case SAB, DOM -> "fim de semana";
  case SEG, TER, QUA,
    QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust · match exaustivo

```
match n {
  132::MIN..=-1 => "negativo",
  0 => "zero",
  1..=9 => "um dígito",
  _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige fallthrough. C#: cair implicitamente é proibido (p. 336). Python 3.10: match com padrões e guardas.

Sebesta, 11ª ed. · §8.2.2 · p. 334-341 · exemplos/07_selecao

Slide 17

Seleção múltipla. No switch de C, sem break o controle "cai" nos casos seguintes (com index = 1 executa odd, even e o erro do default). Java 14+ tem switch com -> que não cai e pode devolver valor; Rust usa match exaustivo, em que faltar um caso é erro de compilação. Em Go cada caso termina sozinho, em C# cair é proibido e Python 3.10 tem match com padrões.

Laços controlados por contador

C - o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
    i += 2;           // altera a contagem  
}  
                        // 0 3 6 9  
  
for (; i <= 100; ) // expressões opcionais  
    soma += i++;
```

Python - o for percorre um iterável

```
for i in range(0, 10):  
    print(i, end=" ")  
    i += 2           # NÃO muda nada  
                        # 0 1 2 3 4 5 6 7 8 9  
  
for i in range(10, 0, -3): # 10 7 4 1  
    ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Sebesta, 11ª ed. - 68.3.1 - p. 342-345 - exemplos/08_iteracao

Slide 18

Laços controlados por contador. O for de C é um laço lógico disfarçado: alterar i dentro do corpo muda a contagem (imprime 0 3 6 9). No for de Python, que percorre um iterável, alterar i não muda nada (imprime 0 a 9). As questões de projeto são o tipo e o escopo da variável, se ela pode ser alterada no corpo e se os parâmetros são avaliados uma vez ou a cada volta.

Laços lógicos, saídas e iteradores

Java - break rotulado (p. 348)

```
externo:
for (int i = 0; i < m.length; i++)
  for (int j = 0; j < m[i].length; j++)
    if (m[i][j] < 0) {
      System.out.println(i + "," + j);
      break externo; // sai dos DOIS laços
    }
```

JavaScript - for...in × for...of

```
const notas = [10, 20, 30];
let total = 0;
for (const i in notas) total += i;
console.log(total); // "0012": chaves!

total = 0;
for (const v of notas) total += v;
console.log(total); // 60: valores
```

Zune (2008) - o laço lógico sem progresso

```
while (dias > 365) {
  if (bissexto(ano)) {
    if (dias > 366) { dias -= 366; ano++; }
    // dias == 366: nada muda, laço infinito!
  } else { dias -= 365; ano++; }
}
```

Pós-teste (do-while) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Sebesta, 11ª ed. · §8.3.2-8.3.4 · p. 345-352 · exemplos/08_iteracao

Slide 19

Laços lógicos, saídas e iteradores. Java tem break rotulado, que sai de dois laços de uma vez. Em JavaScript, for...in percorre as chaves (a soma dá "0012") e for...of percorre os valores (60). O bug do Zune é um laço sem progresso: no dia 366 de um ano bissexto nenhum ramo altera dias e o while nunca termina. O pós-teste (do-while) executa ao menos uma vez, o que é fácil de esquecer.

Desvio incondicional: o goto

"goto fail" (Apple, 2014) · versão simplificada

```
int valida_certificado(void) {  
    int err;  
    if ((err = verifica_hash()) != 0)  
        goto fail;  
    goto fail;           // duplicada: desvia SEMPRE  
    if ((err = verifica_assinatura()) != 0)  
        goto fail;       // nunca é alcançada  
fail:  
    return err;           // 0 = "certificado válido"  
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java **não** tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de indentação enganosa), o bug teria sido pego.

Sebesta, 11ª ed. · §8.4 · p. 352-353 · exemplos/09_desvio_protegido

Slide 20

O desvio incondicional goto. O bug "goto fail" da Apple tinha um goto duplicado, fora do if por falta de chaves, que desviava sempre e pulava a verificação da assinatura, deixando err com 0 (sucesso). Dijkstra criticou o goto em 1968; Java não o tem, enquanto C, C++, C# e Go têm. break, continue e return são gotos restritos que melhoram a legibilidade. Chaves obrigatórias ou o gcc com -Wall teriam pego o bug.

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x = y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no if) ou fim do laço (no do ... od)
- Base do **CSP** de Hoare (1978) e do select de Go

Go · select, herdeiro dos comandos protegidos

```
a <- "a"      // os dois canais
b <- "b"      // estão prontos

select {      // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495] (muda a cada execução)
```

Sebesta, 11ª ed. · §8.5 · p. 353-355 · exemplos/09_desvio_protegido

Slide 21

Comandos protegidos de Dijkstra: todas as guardas são avaliadas; se várias são verdadeiras, a escolha é não determinística; se nenhuma é, é erro (no if) ou fim do laço (no do...od). Quando $x = y$, qualquer ramo serve no exemplo do máximo. Essa ideia é a base do CSP de Hoare e do select de Go, que sorteia uma opção quando vários canais estão prontos (o resultado muda a cada execução).

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

1 · JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
  total += p;
console.log("Total: " + total);
```

2 · Python

```
def desconto(preco, d=None):
    d = d or 10      # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em oncompiler.com

Slide 22

Exercício em duplas (1/3). Caso 1 (JavaScript): imprime "Total: 0012", porque for...in percorre as chaves e o + concatena; corrigir com for...of ou reduce. Caso 2 (Python): desconto(200, 0) dá 180.0 em vez de 200.0, porque "d or 10" trata o 0 como falso; corrigir com "if d is None".

Exercício: encontre o bug (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

Slide 23

Exercício em duplas (2/3). Caso 3 (Java): imprime "inválido", porque o switch sem break cai do case 2 até o default; corrigir com break ou switch com ->. Caso 4 (C): imprime "saldo insuficiente" e "fim" mesmo com saque menor que o saldo, porque o ; logo depois do if é uma sentença vazia; corrigir usando chaves sempre.

Exercício: encontre o bug (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

5 · Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 · Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

Slide 24

Exercício em duplas (3/3). Caso 5 (Go): imprime "dentro: 20" e "fora: 10", porque `:=` dentro do `if` cria uma nova variável `x` (sombreamento); corrigir com `x = x * 2`. Caso 6 (Java): imprime "0.0%", porque a divisão inteira `7 / 10` vale 0 antes do alargamento para `double`; corrigir com `acertos * 100.0 / total`.

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2, 1 + 2 + "3"</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>===</code>
Booleanos	<code>0, "", []</code> são falsos; <code>a > b > c</code> em C	<code>if</code> só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	<code>if (x = y)</code>	atribuição não é expressão (Python, Go, Rust)
Seleção	<code>switch</code> sem <code>break</code> ; <code>else</code> pendente	<code>-></code> , <code>match</code> exaustivo, chaves obrigatórias
Laços	<code>for...in</code> no JS; laço sem progresso	<code>for...of</code> , iteradores, <code>break</code> rotulado
Desvio	<code>goto</code> sem restrição	<code>break</code> , <code>continue</code> , <code>return</code> : gotos restritos

Sebesta, 11ª ed. · Caps. 7 e 8 · p. 299-361

Slide 25

Síntese das armadilhas e de como as linguagens mais novas ajudam: parênteses contra problemas de precedência; Java fixa a ordem de avaliação; Go e Rust exigem conversão explícita; `if` só aceita booleano em Java, Go e Rust; atribuição não é expressão em Python, Go e Rust; `switch` com `->` e `match` exaustivo; `for...of` e `break` rotulado; e `break`, `continue` e `return` como gotos restritos. Cada restrição remove uma classe de bugs, trocando um pouco de facilidade de escrita por confiabilidade.

Atividades complementares (autopesquisa)

- **Expressões em Ruby e em Lisp** (§7.2.1.4-7.2.1.5, p. 305-306):
`expressoes.rb`, `expressoes.lisp`
- **Expressões condicionais** (§7.2.1.6, p. 306) e **erros em expressões** (§7.4.3, p. 313)
- **Implementação da seleção múltipla**: tabelas de desvio (§8.2.2.3, p. 337-339)
- **Laços em linguagens funcionais**: contagem por recursão (§8.3.1.4, p. 345)
- **Iteradores e blocos de Ruby** (§8.3.4, p. 349-352)
- **Leitura**: Dijkstra, "Go To Statement Considered Harmful" (1968)

Entrega sugerida

Um mesmo exemplo com `switch` em C, `switch` -> em Java, `switch` em Go e `match` em Rust: compare "cair" entre casos, exaustividade e legibilidade.

Próxima aula

Subprogramas (Caps. 9 e 10)

Sebesta, 11ª ed. · Caps. 7 e 8 · exemplos/11_autopesquisa

Slide 26

Atividades de autopesquisa: expressões em Ruby e Lisp, expressões condicionais, tabelas de desvio na implementação da seleção múltipla, laços em linguagens funcionais, iteradores e blocos de Ruby, e a leitura do artigo de Dijkstra sobre o goto (1968). Entrega sugerida: um mesmo exemplo de seleção múltipla em C, Java, Go e Rust, comparando queda entre casos, exaustividade e legibilidade. A próxima aula é sobre subprogramas (capítulos 9 e 10).